

安卓 APP 逆向工程

学习指导书

（初学者版）

从零开始，系统地掌握 Android 应用逆向分析

仅供安全研究、学习与合法测试使用

2026 年 8 月 18 日

前言

随着移动互联网的普及，Android 应用已经深入到日常生活的方方面面。对于希望从事移动安全、应用安全测试、恶意软件分析，或者单纯想深入理解一个 App 内部运行原理的读者来说，逆向工程是一项非常实用且充满乐趣的技能。

本书面向**完全没有逆向经验**的初学者，从最基础的概念讲起，循序渐进地介绍 Android 应用逆向工程的完整知识体系。全书不追求晦涩的理论，而是以「看得懂、能上手」为目标，配合大量实例，帮助读者建立属于自己的分析思路和工具链。

学习本书的前提

在开始之前，建议读者具备以下基础（不强求精通）：

- 了解基本的编程概念，最好会用 Java 或 Kotlin 写过简单程序；
- 会使用基本的命令行（Windows 的 PowerShell / CMD，或 Linux 终端）；
- 有一台可以联网的电脑，以及（可选）一台 Android 设备或模拟器。

如何使用本书

- 第 1-2 章是概念与基础，建议通读；
- 第 3 章是环境搭建，务必跟着操作把工具装好；
- 第 4-6 章是核心技能（静态分析、Smali、动态分析），建议边看边动手；
- 第 7 章是综合实战，学完前几章后再练习；
- 第 8 章与附录作为进阶方向与查阅资料。

注意

合规提醒：逆向工程是一把双刃剑。请在法律允许的范围内使用本书所学，只对自己拥有、或已获得明确书面授权的应用进行逆向分析。未经授权破解他人软件、绕过付费验证、盗取他人代码或数据，可能构成侵权甚至违法犯罪。本书的全部内容仅用于安全研究、教学与合法的安全测试目的，请务必遵守当地法律法规与职业操守。

目录

前言	i
第一章 逆向工程概述	1
1.1 什么是逆向工程	1
1.2 Android 逆向工程能做什么	1
1.3 法律与道德边界	2
1.4 整体学习路线	2
第二章 Android 应用技术基础	3
2.1 APK 文件结构	3
2.2 从源码到 APK：打包流程	4
2.3 四大组件与 AndroidManifest.xml	4
2.4 DEX 字节码与 ART/Dalvik	5
2.5 JNI 与 native 层简介	5
第三章 环境搭建	6
3.1 工具清单总览	6
3.2 基础环境：JDK 与 adb	6
3.3 静态分析工具：jadx 与 apktool	7
3.4 动态分析工具：Frida	8
3.5 测试设备：模拟器与真机	8
第四章 静态分析入门	9
4.1 获取目标 APK	9
4.2 用 apktool 解包	9
4.3 用 jadx 反编译 Java 代码	10
4.4 阅读 AndroidManifest.xml	10
4.5 资源与字符串定位	10
4.6 认识混淆与反混淆	10

第五章 Smali 语法快速入门	12
5.1 为什么需要学 Smali	12
5.2 寄存器与类型描述符	12
5.3 基本指令	13
5.4 方法调用与字段访问	13
5.5 一个完整的 Smali 例子	14
第六章 动态分析入门	16
6.1 adb 常用命令	16
6.2 logcat 日志	16
6.3 Frida 原理与安装	17
6.4 第一个 Hook: Java 层函数拦截	17
6.5 实战: 定位关键逻辑	18
第七章 实战案例	19
7.1 案例准备: 一个用于学习的示例	19
7.2 案例一: 静态定位关键逻辑	19
7.3 案例二: 用 Frida 观察函数调用	20
7.4 案例三: 修改 Smali 并重打包	20
7.5 常见坑与排错	21
第八章 进阶方向	22
8.1 native 层 (.so) 逆向	22
8.2 加固与脱壳	22
8.3 协议与网络分析	22
8.4 自动化与工具开发	23
附录 A 常用命令速查	24
附录 B 术语表	26
结语	27

第一章 逆向工程概述

1.1 什么是逆向工程

逆向工程（Reverse Engineering，简称 RE）是指对已有的成品进行分析，反推其设计原理、实现方式和工作流程的过程。与之相对的是「正向开发」——从需求出发编写代码，最终得到可运行的产品。

对于 Android 应用而言，正向开发得到的是一个 **APK** 安装包（本质是一个压缩文件）。逆向工程则是从这个 APK 出发，反推出它的：

- 代码逻辑（通过反编译 DEX 字节码得到 Java 或 Smali 代码）；
- 资源配置（界面布局、字符串、图标等）；
- 网络行为（与服务器通信的接口与协议）；
- 原生实现（编译成机器码的 **.so** 库）。

1.2 Android 逆向工程能做什么

在合法与受控的前提下，逆向工程有非常广泛且正当的应用场景：

- **安全测试（渗透测试 / 安全评估）**：企业请安全工程师对自家 App 进行安全审计，发现漏洞（如敏感信息泄露、签名校验缺失、通信未加密等）；
- **恶意软件分析**：安全研究人员分析病毒、木马、钓鱼 App 的行为，提取其恶意逻辑与攻击基础设施（C2 服务器等）；
- **漏洞研究**：在 CTF 比赛、漏洞挖掘中理解程序逻辑，定位可利用的缺陷；
- **兼容性与调试**：分析第三方 SDK 或闭源库的行为，排查崩溃、性能问题；
- **学习与教学**：通过分析开源或自研应用，深入理解 Android 系统与虚拟机的工作原理。

1.3 法律与道德边界

学习逆向工程之前，必须先把「什么能做、什么不能做」想清楚。以下是通行的基本准则：

- **只分析你有权限的目标：**自己的应用、明确授权的测试目标、公开的 CTF 题目、恶意样本（在隔离环境）。
- **不以牟利或破坏为目的：**不得将逆向成果用于破解付费功能、盗版、窃取用户数据、制作外挂等。
- **尊重知识产权：**反编译得到的代码仍受版权保护，不得复制、分发或用于商业产品。
- **注意授权范围：**即使是安全测试，也应签署授权协议（如渗透测试授权书），明确测试边界。

注意

记住一个简单判断标准：如果这个操作会让你不敢告诉目标应用的开发者，那它大概率是不应该做的。技术本身无罪，关键在使用技术的人。

1.4 整体学习路线

本书建议的学习路线如下，读者可以据此把握学习节奏：

1. 理解 Android 应用的基本结构与运行机制（第 2 章）；
2. 搭建工具环境（第 3 章）；
3. 学会「静态分析」：把 APK 反编译成可读代码并阅读（第 4 章）；
4. 学会阅读和修改 Smali 汇编（第 5 章）；
5. 学会「动态分析」：用 Frida 在运行时观察和干预程序行为（第 6 章）；
6. 通过综合案例把静态与动态分析结合起来（第 7 章）；
7. 根据兴趣向 native 层、加固脱壳等方向深入（第 8 章）。

提示

逆向工程最重要的是建立「读代码 + 动手验证」的循环。每学一个工具，就找一个自己的 App 试一下，效果远好于只看书。

第二章 Android 应用技术基础

2.1 APK 文件结构

APK (Android Package) 本质是一个 ZIP 压缩包，可以使用任意解压工具打开。一个典型的 APK 解压后通常包含以下内容：

```
1 app.apk
2 |— AndroidManifest.xml    # 应用清单（二进制格式，需解码后阅读）
3 |— classes.dex            # DEX 字节码（核心代码，可能有多份 classes2.dex
   ... )
4 |— resources.arsc         # 编译后的资源索引表
5 |— res/                   # 资源目录（布局、图片、字符串等）
6 |— lib/                   # 原生库（按 CPU 架构存放 .so 文件）
   |   |— arm64-v8a/
   |   |— armeabi-v7a/
7 |— assets/                # 原始资源（原样打包，代码可通过 AssetManager
   读取）
8 |— META-INF/              # 签名与证书信息
9 |— kotlin/                # Kotlin 元数据（若使用 Kotlin 开发）
```

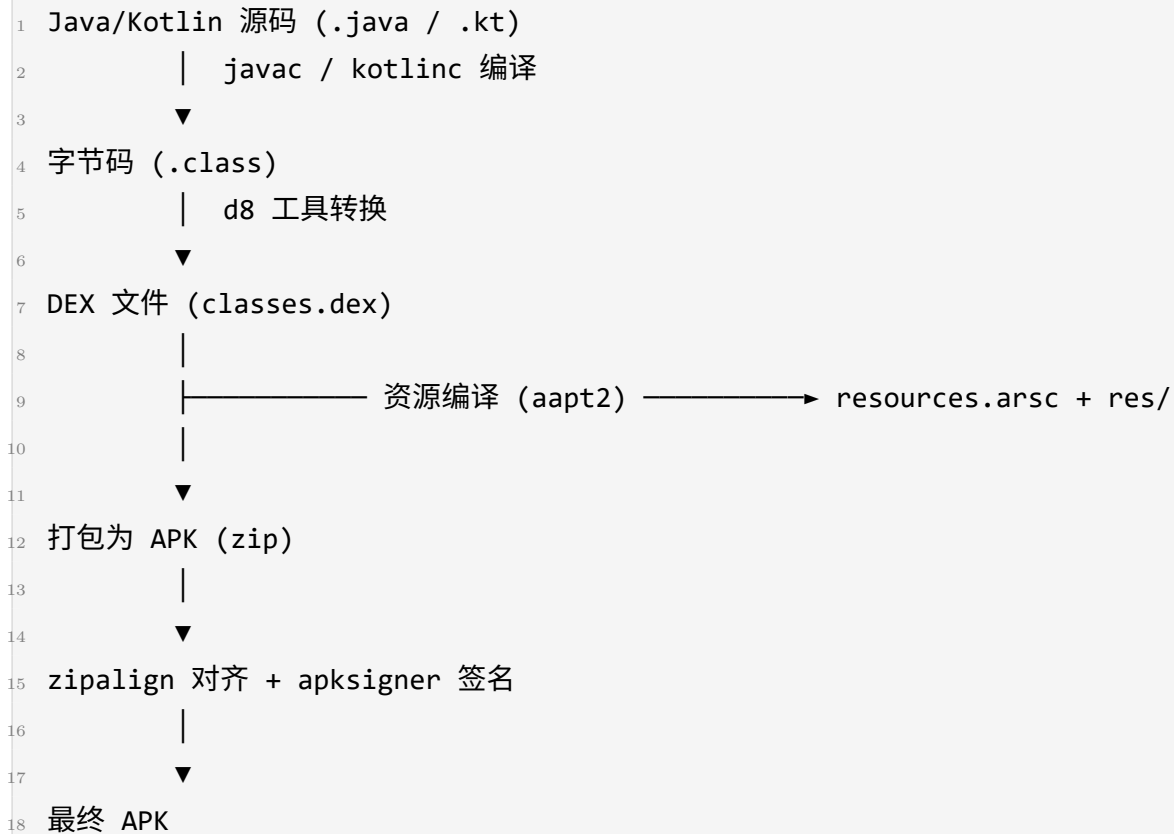
Listing 2.1: APK 解压后的典型结构

各部分的含义简要说明如下：

- **AndroidManifest.xml**: 应用的「身份证」，声明了组件、权限、入口 Activity、包名等关键信息，是逆向分析的首要入手点；
- **classes.dex**: Android 虚拟机可执行的字节码，所有 Java/Kotlin 代码最终都编译到这里，是逆向分析的核心目标；
- **resources.arsc** 与 **res/**: 应用界面与文案资源；
- **lib/**: 使用 JNI (Java Native Interface) 编写的原生代码编译成的动态库，通常用 C/C++ 编写，逆向难度更高；
- **META-INF/**: 包含应用签名文件，用于校验 APK 完整性。

2.2 从源码到 APK：打包流程

理解打包流程有助于理解「逆向」每一步对应的「正向」环节。Android 应用的典型构建过程如下：



Listing 2.2: Android 应用的典型构建流程（简化）

从这个流程可以看出，逆向分析本质上是在反推：拿到 `classes.dex` 后，用反编译工具把它还原成接近源码的 Java 代码，或用反汇编工具得到中间层的 Smali 代码。

2.3 四大组件与 AndroidManifest.xml

Android 应用由若干「组件」构成，理解组件有助于快速定位入口。四大组件分别是：

- **Activity（活动）**：用户能看到的界面，每个界面通常对应一个 Activity；
- **Service（服务）**：后台运行的逻辑，没有界面；
- **BroadcastReceiver（广播接收器）**：接收系统或应用广播消息；
- **ContentProvider（内容提供者）**：对外提供数据共享。

其中，‘AndroidManifest.xml’ 中用 **android:exported** 标记组件是否可被外部调用，逆向分析时常据此判断攻击面。入口 Activity 由 **MAIN** 动作与 **LAUNCHER** 类别标识，是启动应用后最先执行的界面。

2.4 DEX 字节码与 ART/Dalvik

Android 上运行 Java/Kotlin 代码的「虚拟机」经历了两个阶段：

- **Dalvik (旧)**：基于寄存器的虚拟机，解释执行 DEX 字节码，Android 4.4 及以前为主；
- **ART (新)**：Android 5.0 起取代 Dalvik，安装时或运行时将 DEX 预编译为机器码 (AOT) 以提高性能。

DEX (Dalvik Executable) 是一种紧凑的字节码格式，与标准 JVM 的 **.class** 文件不同：它把多个类合并到同一个文件中，并共享常量池，以减小体积。逆向时，我们主要面对的就是这种字节码。Smali 则是 DEX 字节码的「文本化汇编表示」，第 5 章会详细介绍。

2.5 JNI 与 native 层简介

JNI (Java Native Interface) 允许 Java 代码调用 C/C++ 编写的原生函数。开发者常把核心逻辑、加密算法或校验逻辑放进 **.so** 库中，以增加逆向难度。native 方法在 Java 层只有声明 (关键字 **native**)，真正的实现在 **.so** 文件中，需要借助反汇编工具 (如 Ghidra、IDA) 才能分析。这部分属于进阶内容，见第 8 章。

第三章 环境搭建

3.1 工具清单总览

下表列出了本书涉及的主要工具及其用途：

工具	类别	用途
JDK	基础环境	提供 Java 运行环境与 keytool 等工具
Android SDK Platform-Tools	基础环境	提供 adb 调试桥
jadx	静态分析	将 DEX 反编译为 Java 代码
apktool	静态分析	将 APK 解码为 Smali 与资源，可重打包
Frida	动态分析	运行时 Hook（拦截/修改函数）
Ghidra / IDA	高级反汇编	分析 native 层 .so 文件
Android 模拟器 / 真机	测试设备	运行被分析的应用

3.2 基础环境：JDK 与 adb

安装 JDK

下载并安装 OpenJDK 或 Oracle JDK（建议 JDK 17 及以上），安装后确认可用：

```
1 java -version
2 keytool -version
```

Listing 3.1: 验证 JDK 安装

安装 adb（Android Debug Bridge）

adb 是连接电脑与 Android 设备的调试桥梁，几乎贯穿整个逆向流程。可以从 Android 官方 SDK 的 Platform-Tools 中获得，下载后把目录加入系统 **PATH**，然后验证：

```
1 adb version
2 adb devices          # 列出已连接的设备
```

Listing 3.2: 验证 adb 安装

3.3 静态分析工具：jadx 与 apktool

jadx

jadx 能将 DEX 字节码反编译为接近源码的 Java 代码，是最常用的快速阅读工具。它同时提供命令行版本 `jdx` 与图形界面 `jdx-gui`。基本用法：

```
1 # 反编译整个 APK 到 out 目录
2 jadx -d out app.apk
3
4 # 或直接打开图形界面
5 jadx-gui app.apk
```

Listing 3.3: jadx 基本用法

apktool

apktool 将 APK 解码为 Smali 代码与可读资源，并且支持解码后重新打包，常用于「修改代码再回编译」的场景。基本用法：

```
1 # 解码 APK 到 out 目录
2 apktool d app.apk -o out
3
4 # 重新打包 out 目录为新的 APK
5 apktool b out -o rebuilt.apk
```

Listing 3.4: apktool 基本用法

说明

jadx 和 apktool 的分工可以简单理解为：想快速读逻辑用 jadx（Java 代码），想精细修改并重打包用 apktool（Smali 代码）。两者互为补充。

3.4 动态分析工具：Frida

Frida 是一个强大的动态插桩（instrumentation）框架，可以在程序运行时动态注入代码、拦截函数调用、修改返回值。它由两部分组成：

- **服务端（frida-server）**：运行在 Android 设备上，需要与设备架构匹配；
- **客户端（frida-tools）**：运行在电脑上，通过 Python 或命令行与设备交互。

安装客户端：

```
1 pip install frida-tools
2 frida --version
```

Listing 3.5: 安装 Frida 客户端

服务端的部署详见第 6 章。

3.5 测试设备：模拟器与真机

逆向分析需要一个可运行目标 App 的环境：

- **真机**：更接近真实环境，部分 App 会检测模拟器。若需使用 Frida 注入，通常需要 root；
- **模拟器**：如 Android Studio 自带模拟器、Genymotion 等，方便随时创建/还原快照，适合初学。

提示

初学阶段，建议先使用**模拟器**：可以轻松获得 root、安装任意架构的 frida-server、并随时还原环境，降低试错成本。

第四章 静态分析入门

4.1 获取目标 APK

分析的第一步是拿到 APK 文件。常见的合法来源包括：

- 自己编写的应用；
- 从设备中导出已安装的应用（用 `adb` 找到包路径后 `pull` 出来）；
- 从应用商店下载的公开 APK（注意版权与授权）；
- 公开的 CTF 题目或教学样例。

从已连接设备导出某个已安装应用的 APK 示例：

```
1 # 找到应用的安装路径
2 adb shell pm path com.example.demo
3 # 输出形如：package:/data/app/com.example.demo-xxx/base.apk
4
5 # 拉取到本地
6 adb pull /data/app/com.example.demo-xxx/base.apk ./demo.apk
```

Listing 4.1: 从设备导出 APK

4.2 用 apktool 解包

拿到 APK 后，先用 apktool 解码，观察其结构：

```
1 apktool d demo.apk -o demo
2 # 之后可以在 demo/ 目录下看到
3 #   AndroidManifest.xml （已还原为可读文本）
4 #   smali/ smali_classes2/ ... （Smali 代码）
5 #   res/      （资源）
```

Listing 4.2: 解码 APK 并查看结构

解码后，`AndroidManifest.xml` 已经变成可读的文本，可以直接用编辑器打开。

4.3 用 jadx 反编译 Java 代码

Smali 代码可读性较差，初学阶段更推荐先用 jadx 看 Java 代码：

```
1 jadx -d demo_java demo.apk
2 # 打开 demo_java/ 目录，即可浏览接近源码的 Java 代码
```

Listing 4.3: jadx 反编译

提示

逆向时先读 jadx 的 Java 代码建立整体理解，再在需要精确定位/修改时回到 apktool 的 Smali 代码。这个顺序能让新手少走很多弯路。

4.4 阅读 AndroidManifest.xml

反编译后首先要看的是清单文件。重点信息包括：

- **包名**（`package` 属性）：应用的唯一标识，后续工具会频繁用到；
- **入口 Activity**：找到 `android.intent.action.MAIN` 与 `android.intent.category.LAUNCH` 所在的 Activity，这是分析起点；
- **权限声明**：`uses-permission` 列出了应用申请的权限，可据此猜测其功能（如申请短信权限可能涉及短信收发逻辑）；
- **exported 组件**：标记为 `android:exported="true"` 的组件可能被外部调用，是常见的安全风险点。

4.5 资源与字符串定位

字符串是逆向分析的重要线索。通过搜索可读字符串，往往能快速定位关键逻辑：

- 在 jadx-gui 中按 **Ctrl+Shift+F**（全局搜索）搜索关键词；
- 常见搜索目标：错误提示、Toast 文案、URL、接口路径（如 `/api/login`）、密钥片段等；
- 定位到字符串被引用的位置后，向上回溯调用链即可找到核心逻辑。

4.6 认识混淆与反混淆

为了保护代码，很多应用会开启混淆（ProGuard / R8）。混淆后的代码特征是：类名、方法名变成 `a`、`b`、`aaa` 这样的短名称，可读性大幅下降。应对策略：

- 优先借助字符串、资源引用定位：混淆通常不会改动字符串常量与资源名；
- 关注系统 API 调用：混淆不改动框架方法名（如 `Toast.show`），可据此推断代码意图；
- 耐心回溯调用链：从已知的「锚点」（如入口 Activity、按钮事件）逐步追查。

说明

混淆只增加阅读难度，不改变程序的行为。只要方法还在，逻辑就还在，只是名字难看了而已。

第五章 Smali 语法快速入门

5.1 为什么需要学 Smali

jadx 反编译出的 Java 代码虽然好读，但有两个局限：

- 遇到强混淆时，jadx 可能反编译失败或产出不完整的代码；
- 若要修改应用并重新打包，必须直接改 Smali 代码（apktool 的工作单位就是 Smali）。

因此，掌握 Smali 的阅读与基本修改能力，是从「能看」迈向「能改」的关键一步。

5.2 寄存器与类型描述符

Smali 使用寄存器来存值。在方法中：

- `v0`、`v1`、`v2...` 表示局部寄存器；
- `p0`、`p1`、`p2...` 表示方法参数（`p0` 在非静态方法中通常是 `this`）。

Smali 使用紧凑的类型描述符表示类型，常用对照如下：

描述符	Java 类型	说明
V	void	无返回值
Z	boolean	布尔
B	byte	字节
S	short	短整型
C	char	字符
I	int	整型
J	long	长整型
F	float	单精度浮点
D	double	双精度浮点
Ljava/lang/String;	String	对象类型，以 L 开头；结尾
[I	int[]	数组，以 [开头

5.3 基本指令

Smali 的指令可以按功能分类，下面是最常用的几类：

- 数据移动：move、const、const-string（加载字符串常量）；
- 方法调用：invoke-virtual、invoke-direct、invoke-static、invoke-super、invoke-interface；
- 返回：return（返回对象）、return-void（无返回值）、return v0（返回寄存器中的值）；
- 字段访问：iget/iput（实例字段读写）、sget/sput（静态字段读写）；
- 条件跳转：if-eqz（等于 0 则跳）、if-nez（不等于 0 则跳）等。

5.4 方法调用与字段访问

Smali 的方法调用使用「类名 → 方法名 (参数描述符) 返回描述符」的格式。例如：

```

1  # 调用静态方法：android.util.Log.d(String, String) 返回 int
2  invoke-static {v0, v1}, Landroid/util/Log;->d(Ljava/lang/String;Ljava/
   lang/String;)I
3
4  # 读取字段：this.name 为 String 类型
5  iget-object v0, p0, Lcom/example/demo/MainActivity;->name:Ljava/lang/
   String;
```

Listing 5.1: Smali 方法调用示例

5.5 一个完整的 Smali 例子

下面用一个完整的 Activity 说明 Smali 的整体结构。假设其 Java 源码为：

```
1 public class MainActivity extends Activity {  
2     private boolean isVip = false;  
3  
4     private boolean checkVip() {  
5         return isVip;  
6     }  
7 }
```

Listing 5.2: 对应的 Java 源码

反汇编后的 Smali 大致如下：

```
1 .class public Lcom/example/demo/MainActivity;  
2 .super Landroid/app/Activity;  
3  
4 # 实例字段声明  
5 .field private isVip:Z  
6  
7 # 构造方法  
8 .method public constructor <init>()V  
9     .locals 1  
10    invoke-direct {p0}, Landroid/app/Activity;-><init>()V  
11    return-void  
12 .end method  
13  
14 # checkVip() 方法  
15 .method private checkVip()Z  
16     .locals 1  
17     iget-boolean v0, p0, Lcom/example/demo/MainActivity;->isVip:Z  
18     return v0  
19 .end method
```

Listing 5.3: 对应的 Smali 代码（简化）

说明

对照 Java 源码与 Smali 阅读，可以快速掌握二者对应关系。重点理解：**.method** 声明方法、**.locals** 声明寄存器数量、**iget/iput** 读写字段、**return** 返回。

第六章 动态分析入门

6.1 adb 常用命令

动态分析离不开 adb。下表汇总了最常用的命令：

命令	作用
adb devices	列出连接的设备
adb install app.apk	安装应用
adb uninstall < 包名 >	卸载应用
adb shell	进入设备 shell
adb shell am start -n 包名/Activity	启动指定 Activity
adb shell pm list packages	列出所有包名
adb logcat	查看系统与应用日志
adb pull /path/file 本地路径	从设备拉取文件
adb push 本地文件 /path/	向设备推送文件

6.2 logcat 日志

logcat 是分析运行行为的第一手资料。很多应用会把关键信息打印到日志中：

```
1 # 实时查看全部日志
2 adb logcat
3
4 # 只查看某个 tag 的日志
5 adb logcat -s MyTag
6
7 # 清空日志
8 adb logcat -c
```

Listing 6.1: 查看日志并过滤

提示

逆向时，在关键操作前后对比 logcat 输出，往往能直接发现相关的类名、方法名或错误信息，是「定位问题」的捷径。

6.3 Frida 原理与安装

Frida 的核心思想是：把一段脚本注入到目标进程，在运行时动态地拦截（Hook）任意函数。它工作于应用之外，无需修改 APK 本身，非常适合观察程序的真实行为。

部署 frida-server

1. 到 Frida 官方仓库下载与设备 CPU 架构匹配的 frida-server（如 arm64、x86_64）；
2. 解压后推送到设备并赋予执行权限；
3. 以 root 权限启动服务端。

```
1 adb push frida-server /data/local/tmp/
2 adb shell "chmod 755 /data/local/tmp/frida-server"
3 adb shell "su -c '/data/local/tmp/frida-server &'"
```

Listing 6.2: 部署并启动 frida-server

启动后另开终端验证连接：

```
1 frida-ps -U          # 列出设备上的进程，说明连接成功
```

Listing 6.3: 验证 Frida 连接

6.4 第一个 Hook：Java 层函数拦截

下面是一个最小可用的 Frida 脚本，用于拦截并修改 Java 层方法的行为：

```
1 // 在 Java 虚拟机中执行
2 Java.perform(function () {
3     // 定位目标类
4     var MainActivity = Java.use("com.example.demo.MainActivity");
5
6     // 替换 checkVip 方法的实现
7     MainActivity.checkVip.implementation = function () {
```

```
8     console.log("[*] checkVip() 被调用");
9     var original = this.checkVip();    // 调用原方法
10    console.log("[*] 原返回值: " + original);
11    return true;                        // 强制返回 true
12  };
13 });
```

Listing 6.4: hook.js ——拦截 checkVip 方法

通过 `-f` 参数在应用启动时注入，或通过 `-n` 附加到已运行的进程：

```
1 # 启动应用并注入脚本
2 frida -U -f com.example.demo -l hook.js --no-pause
3
4 # 或附加到正在运行的进程
5 frida -U -n com.example.demo -l hook.js
```

Listing 6.5: 注入脚本

注意

上述示例只是为了演示 Frida 的拦截与改写能力，请只在你自己的或已授权的应用上练习。Hook 本身不修改 APK 文件，但行为上等同于临时改变程序逻辑，同样受法律与授权约束。

6.5 实战：定位关键逻辑

动态分析的典型流程如下：

1. 用 `adb` 启动目标应用，配合 `logcat` 观察行为；
2. 用 `jadx` 静态定位可疑方法（如包含 `check`、`verify`、`isVip` 等关键字）；
3. 用 Frida Hook 这些方法，打印参数与返回值，验证假设；
4. 根据观察结果，决定是否需要进一步修改逻辑（进入 Smali 修改与重打包流程）。

提示

静态分析画地图，动态分析验真假。两者结合才能高效、准确地理解程序。

第七章 实战案例

本章通过一个综合案例，把前面学到的静态分析、Smali 修改与动态分析串起来。请务必使用自己编写的 Demo 应用或已获授权的目标进行练习。

7.1 案例准备：一个用于学习的示例

建议读者自己编写一个最简单的 Android Demo 应用作为练习对象，例如包含一个返回布尔值的方法：

```
1 public class MainActivity extends Activity {  
2     // 模拟一个「是否付费用户」的判断  
3     private boolean checkVip() {  
4         return false;  
5     }  
6 }
```

Listing 7.1: 练习用的 Demo 源码（节选）

把该应用编译成 APK 后，它就是本书所有练习的「靶子」。

7.2 案例一：静态定位关键逻辑

目标：从 APK 出发，找到 checkVip 方法。

1. 用 jadx 反编译 APK，打开 MainActivity.java，直接看到 checkVip 方法；
2. 观察其逻辑：无条件返回 false；
3. 用 apktool 解码，在 smali/ 目录下找到同名 .smali 文件，定位到对应方法体。

```
1 .method private checkVip()Z  
2     .locals 1  
3     const/4 v0, 0x0          # 0x0 即 false  
4     return v0
```

```
5 .end method
```

Listing 7.2: checkVip 的 Smali 代码

7.3 案例二：用 Frida 观察函数调用

目标：运行时观察 `checkVip` 的返回值，验证静态分析结论。

使用第 6 章的 `hook.js` 脚本注入后，日志会显示：

```
1 [*] checkVip() 被调用
2 [*] 原返回值: false
```

Listing 7.3: Frida 输出示例

这与静态分析结论一致，说明定位正确。

7.4 案例三：修改 Smali 并重打包

目标：在 Smali 层修改 `checkVip` 的返回值，观察行为变化（仅用于理解原理）。

1. 用 `apktool` 解码 APK；
2. 把 `const/4 v0, 0x0` 改为 `const/4 v0, 0x1`（`0x1` 即 `true`）；
3. 重新打包；
4. 重新签名（修改后的 APK 原签名失效，必须重新签名才能安装）；
5. 安装并验证行为变化。

```
1 # 重新打包
2 apktool b demo -o rebuilt.apk
3
4 # 生成签名（首次需要）
5 keytool -genkey -v -keystore test.keystore -alias test -keyalg RSA -
   validity 3650
6
7 # 用 apksigner 签名（先做 zipalign 对齐再签名更规范）
8 zipalign -p 4 rebuilt.apk aligned.apk
9 apksigner sign --ks test.keystore --ks-pass pass:test123 --key-pass pass
   :test123 \
10    --out signed.apk aligned.apk
```

```

11
12 # 安装并运行
13 adb install signed.apk
    
```

Listing 7.4: 重打包与重签名流程

注意

重打包与重签名是逆向工程中「修改程序」的核心手段，也正因如此，它最容易越过法律红线。请严格将其用于：理解签名校验原理、安全测试、教学演示，而绝不用于破解、盗版或绕过付费。

7.5 常见坑与排错

初学阶段容易遇到的问题及解决办法：

- **重新签名后 App 闪退**：很可能是应用有**签名校验**（运行时校验自身签名），这是常见保护手段，需要 Hook 或 patch 掉校验逻辑；
- **adb 无法连接设备**：检查设备是否开启 USB 调试，运行 `adb kill-server` 后 `adb start-server` 重试；
- **frida-server 无法启动**：确认架构匹配、设备已 root、su 可用；
- **jadx 反编译结果混乱**：属于正常现象，切换到 apktool 看 Smali，或结合动态分析定位。

第八章 进阶方向

掌握基础之后，可以根据兴趣选择深入方向：

8.1 native 层（.so）逆向

当核心逻辑被放进 .so 库时，Java 层只能看到 native 声明。此时需要：

- 用 Ghidra（免费）或 IDA（商业）反汇编 .so 文件；
- 结合 JNI 函数签名（如 `Java_com_example_demo_MainActivity_checkVip`）定位对应函数；
- 用 Frida 的 `Interceptor.attach` 在 native 层 Hook。

8.2 加固与脱壳

许多应用使用商业加固（壳）来隐藏真实代码，APK 中看到的是壳代码，运行时才在内存中解密还原真实 DEX。相关技术包括：

- **脱壳（Dump）**：在运行时从内存中 dump 出被还原的 DEX；
- **自动化脱壳工具**：如 Frida 脚本、FART 等；
- **分析壳行为**：理解壳的加载、解密与反调试机制。

8.3 协议与网络分析

很多 App 的核心在服务端，客户端逆向常与网络分析结合：

- 使用抓包工具（如 mitmproxy、Charles）观察 HTTP(S) 请求；
- 处理 SSL Pinning（证书校验），通过 Frida 绕过以观察加密流量；
- 分析请求的加密签名算法（逆向其 `sign` 参数生成逻辑）。

8.4 自动化与工具开发

熟练后可以自己写工具提高效率：

- 用 Frida 的 Python 绑定编写自动化 Hook 脚本；
- 编写脱壳、批量扫描、字符串提取等辅助工具；
- 参与开源项目（如 Frida、jadx）贡献代码。

附录 A 常用命令速查

A.1 基础与调试

```
1 adb devices          # 列出设备
2 adb install app.apk   # 安装
3 adb shell pm list packages # 列出包
4 adb shell am start -n pkg/Act # 启动 Activity
5 adb logcat -s TAG     # 查看日志
```

A.2 静态分析

```
1 apktool d app.apk -o out      # 解码为 Smali + 资源
2 apktool b out -o rebuilt.apk  # 重新打包
3 jadx -d out app.apk          # 反编译为 Java
4 jadx-gui app.apk             # 图形界面
```

A.3 动态分析

```
1 frida-ps -U              # 列出设备进程
2 frida -U -f pkg -l hook.js # 启动并注入
3 frida -U -n pkg -l hook.js # 附加并注入
```

A.4 重签名

```
1 keytool -genkey -v -keystore k.keystore -alias a -keyalg RSA -validity
  3650
2 zipalign -p 4 rebuilt.apk aligned.apk
```

```
3 apksigner sign --ks k.keystore --out signed.apk aligned.apk
```

附录 B 术语表

术语	解释
APK	Android 应用安装包格式
DEX	Android 虚拟机可执行字节码格式
Smali	DEX 字节码的文本化汇编表示
ART	Android 运行时（取代 Dalvik 的虚拟机）
JNI	Java 原生接口，用于调用 C/C++ 代码
adb	Android 调试桥
Hook	运行时拦截/修改函数行为的技术
签名	用于验证 APK 完整性与来源的机制
混淆	重命名代码以增加阅读难度的保护手段
加固	通过壳保护应用，运行时解密真实代码
SSL Pinning	客户端校验服务器证书，防止中间人抓包

结语

恭喜你完成了这份学习指导书。你已经掌握了 Android 逆向工程的核心概念与工具链：从理解 APK 结构，到静态分析、阅读 Smali、动态 Hook，再到综合实战。逆向工程的魅力在于，它是一个「分析—验证—再分析」的持续探索过程，需要耐心与好奇心。

请始终牢记：**技术是工具，能力越大，责任越大**。把这项能力用于保护应用安全、守护用户隐私、打击恶意软件，而不是用于破坏与牟利。希望你在安全研究的道路上走得更远。